

최규원_종합실습_Transformer 기반 미니 언어모델 개발/해석 보고서

1. Transformer 구조 이해

본 실습에서는 김유정의 단편 작품 8편을 학습 데이터로 사용하여 문자 단위 GPT 언어모델을 구현하고, Transformer가 문맥을 학습하여 다음 문자를 예측하는 과정을 확인하였다. 전체 데이터는 97,708자이며, 서로 다른 문자는 공백과 문장부호를 포함하여 총 1,271개이다.

1.1 토큰화와 다음 문자 예측

이 모델은 단어가 아니라 문자 하나를 하나의 토큰으로 사용한다. 예를 들어 **오**, **늘**, **도**, 공백은 각각 서로 다른 정수로 변환된다. 문자에 부여된 정수는 계산을 위한 식별 번호일 뿐이며, 숫자의 크기 자체가 문자의 의미를 나타내지는 않는다.

입력 x와 정답 y는 다음과 같이 한 글자씩 이동한 관계로 구성된다.

- 입력 x: **오늘도 또 우리 수탉이 막 쫓기었다.**
- 정답 y: **늘도 또 우리 수탉이 막 쫓기었다.**

따라서 모델은 입력 문장 전체에 대해 하나의 정답만 예측하는 것이 아니라, 각 위치에서 바로 다음에 등장할 문자를 예측한다. 예를 들어 입력의 첫 번째 문자인 **오**를 보고 **늘**을 예측하고, **오늘**까지 확인한 위치에서는 다음 문자 **도**를 예측하는 방식이다.

1.2 Token Embedding과 Position Embedding

정수로 변환된 문자는 **Token Embedding**을 통해 192차원 벡터로 변환된다. 이 벡터에는 학습을 통해 얻은 문자의 특성이 담긴다. 그러나 Token Embedding만 사용하면 같은 문자가 문장의 어느 위치에 등장했는지 구분할 수 없다. 이를 보완하기 위해 각 문자의 위치를 나타내는 **Position Embedding**을 더한다.

입력 벡터 = Token Embedding + Position Embedding

따라서 같은 **오**라는 문자라도 첫 번째 위치에 등장한 경우와 열 번째 위치에 등장한 경우에는 서로 다른 최종 입력 벡터가 만들어진다. 이를 통해 모델은 문자 정보뿐만 아니라 문자의 순서도 함께 처리할 수 있다.

1.3 Self-Attention과 Q·K·V

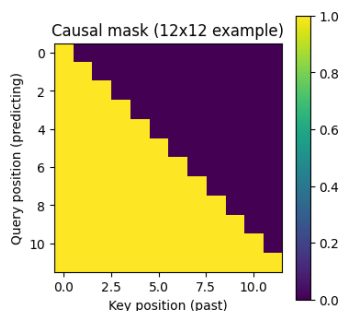
Self-Attention은 현재 문자를 처리할 때 앞 문맥의 어느 부분을 얼마나 참고할지 계산하는 구조이다. 입력 벡터는 서로 다른 가중치 행렬을 통과하여 Q, K, V로 변환된다.

- **Q(Query):** 현재 위치에서 찾고자 하는 정보이다.
- **K(Key):** 각 위치가 어떤 정보를 가지고 있는지를 나타내는 표지이다.
- **V(Value):** 해당 위치에서 실제로 전달할 정보이다.

현재 위치의 Q와 앞에 있는 각 문자의 K를 비교하면 문자 사이의 관련도 점수가 계산된다. 점수가 높을수록 해당 문자의 V가 현재 위치의 표현에 더 많이 반영되도록 가중치가 부여된다.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \times V$$

본 모델은 192차원을 6개의 Attention Head로 나누므로 각 **Head는 32차원을 처리**한다. 여러 Head를 사용하면(**Multi-Head Attention**) 가까운 문자 관계, 멀리 떨어진 문자 관계 등 서로 다른 종류의 문맥 관계를 동시에 학습할 수 있다. 다만 특정 Head가 반드시 문법이나 인물 관계와 같은 하나의 역할만 담당한다고 단정할 수는 없다.



또한 다음 문자를 예측할 때 미래의 정답을 미리 참고하지 못하도록 **Causal Mask**를 적용한다. 이에 따라 각 위치는 현재 위치와 이전 위치만 볼 수 있으며, 마스크 처리된 미래 위치의 점수는 $-\infty$ 가 된다. Softmax를 적용하면 해당 위치의 확률이 0이 되므로 모델이 미래 문자를 참고할 수 없다.

1.4 Transformer Block과 최종 출력

하나의 Transformer Block은 Self-Attention, MLP, LayerNorm, Residual Connection으로 구성되며, 본 모델에서는 총 6개의 Block을 사용한다.

- **Self-Attention**은 앞 문맥에서 필요한 정보를 가져오는 역할을 한다.
- **MLP**는 Attention으로 수집한 정보를 각 문자 위치에서 다시 가공한다.
- **LayerNorm**은 벡터값의 분포를 정돈하여 학습을 안정화한다.

- **Residual Connection**은 기존 벡터에 새로 계산한 정보를 더하여 원래 정보를 보존한다.

MLP에서는 벡터의 크기를 192차원 → 768차원 → 192차원으로 변환한다. 768차원으로 확장하는 과정에서 다양한 특징을 조합하고, 다시 192차원으로 줄여 다음 Block에 전달한다.

6개의 Transformer Block을 통과한 결과는 Language Model Head를 거쳐 각 위치마다 Vocabulary의 1,271개 문자에 대한 Logits로 변환된다. 실제 출력 크기는 (1, 128, 1271)로 나타났다. 이는 한 개의 입력에 포함된 128개 위치에서 각각 1,271개 다음 문자 후보의 점수를 계산했다는 의미이다.

2. 학습 및 생성 과정 이해

2.1 실행 환경과 모델 설정

항목	설정 및 결과
실행 환경	Google Colab GPU, CUDA 활성화
FAST_RUN	False
학습 반복 수	2,000회
학습 시간	229.9초, 약 3분 50초
모델 파라미터 수	약 294만 개
Transformer Block	6개
Attention Head	6개
Embedding 차원	192차원
최대 문맥 길이	128문자
생성 설정	temperature=1.0, top_k=10, 확률적 샘플링

2.2 Loss 변화

모델은 예측한 다음 문자 확률과 실제 정답을 Cross-Entropy Loss로 비교하고, **Loss가 작아지는 방향으로 가중치를 수정한다.**

학습 전 Loss인 7.1354는 1,271개 문자를 거의 동일한 확률로 예측할 때의 이론값인 $\ln(1271) \approx 7.1476$ 과 비슷하다. 이는 학습 전 모델이 다음 문자에 관한 정보를 거의 가지고 있지 않았음을 의미한다.

학습 시점	Training Loss
학습 전	7.1354
iter 0	7.13571
iter 500	2.93729
iter 1,000	2.15553
iter 1,500	1.26911
2,000회 학습 후	0.82754

학습이 진행되면서 Loss는 전체적으로 **7.13571에서 0.82754까지 감소**하였다. 일부 구간에서는 미니배치에 포함된 데이터가 달라져 Loss가 일시적으로 상승하였지만, 전체적인 추세는 꾸준히 감소하였다. 이를 통해 모델이 학습 데이터에 나타난 다음 문자 패턴을 점차 더 잘 예측하게 되었음을 확인하였다.

다만 이는 **Training Loss**이므로 새로운 문장에 대한 일반화 성능까지 향상되었다고 단정할 수는 없다.

2.3 텍스트 생성 과정

모델은 한 번에 완성된 문장을 만드는 것이 아니라 아래의 과정을 반복하여 한 문자씩 생성한다.

1. 시작 문맥을 모델에 입력함.
2. 마지막 위치에서 다음 문자 1,271개의 Logits를 계산함.
3. Softmax를 통해 Logits를 확률로 변환함.
4. 확률이 높은 상위 10개 문자 중 다음 문자 하나를 선택함.
5. 선택한 문자를 기존 문맥 뒤에 붙임.
6. 새로운 문맥을 다시 입력하여 다음 문자를 예측함.

문맥이 128자를 초과하면 가장 최근 128자만 사용한다. 또한 확률적 샘플링을 사용하였기 때문에 같은 모델과 시작 문장을 사용하더라도 실행할 때마다 생성 결과가 달라질 수 있다.

3. 실습 결과 해석

3.1 학습 전·중간·후 생성 결과 비교

→ **학습 전 생성 결과는 다음과 같았다.**

오늘도 또 우리 수탉이 막 쫓알잠떠단셋필쫓켓컨열진망통음떠핑땀알옥존컷티윙옥괭꺽음...

학습 전에는 한글 문자가 출력되었지만, 실제 단어나 문장으로 보기 어려운 **무작위 문자 조합**이 대부분이었다. 띄어쓰기와 문장부호도 불규칙하였다.

→ **500회 학습 후에는 다음과 같은 결과가 생성되었다.**

오늘도 또 우리 수탉이 막 쫓는다는 것이 말이다...

“내가 그만 그렇게 그런 거 어째요.”

이 시점부터 띄어쓰기, 따옴표, 마침표와 같은 문장 형식이 나타났으며 **내가**, **아내**, **사람**, **그렇게** 와 같은 실제 단어도 생성되었다. 그러나 **하고**, **말**, **것이** 와 같은 표현이 반복되고 문장의 의미 연결은 여전히 불분명하였다.

→ **1,000회 이후에는 다음과 같이 대화문과 서술문의 구분이 더 분명해졌다.**

오늘도 또 우리 수탉이 막 쫓기고 있는 것이다.

“내 이자식! 이구, 그래!”

1,500회 결과에서는 **장인님**, **계집애**, **사위** 등 학습 작품에서 자주 등장했을 것으로 보이는 어휘와 인물 관계가 나타났다. 이를 통해 모델이 단순한 문자 배열뿐만 아니라 학습 데이터의 어휘와 문체적 특징도 일부 학습하였음을 확인하였다.

→ **최종 학습 후 생성문은 다음과 같이 시작하였다.**

오늘도 또 우리 수탉이 막 쫓기고 씹이 난다. 장인님 앞으로 우리 쓰는 고 것은 그래에다 엉키고 몸도 못하고 말을 시피를 붙여 가지고...

첫 문장인 **오늘도 또 우리 수탉이 막 쫓기고 씹이 난다.** 는 학습 전 결과보다 문법과 의미가 자연스럽다. 문단 구분, 대화문, 서술형 종결 표현도 나타났다.

최종 생성문은 문장부호와 띄어쓰기, **장인님**, **수탉**, **씹** 과 같은 학습 작품의 어휘를 사용하여 겉보기에는 한국어 문장과 비슷한 형태를 보였으나, 전체 내용을 살펴보면 자연스러운 문장이라고 보기 어렵다. 첫 문장인 **오늘도 또 우리 수탉이 막 쫓기고 씹이 난다.** 는 대략적인 상황을 추측할 수 있지만, 누가 누구에게 쫓기는지 명확하지 않다. 이어지는 **우리 쓰는 고 것은, 그래에다, 말을 시피를 붙여 가지고** 와 같은 표현은 단어와 조사가 잘못 결합되어 문법적으로 성립하지 않으며, 문장 전체의 의미도 파악하기 어렵다. 특히 앞 문장과 뒤 문장 사이의 사건이나 인과관계가 연결되지 않는다. 따라서 모델이 자주 등장하는 문자 배열, 단어, 문장 종결 형식은 학습하였지만, 각 단어의 의미와 문장 전체의 논리적 관계까지 이해한 것은 아니라고 해석하였다.

따라서 모델은 짧은 범위의 문자 배열과 문체는 학습하였지만, 긴 글 전체의 의미나 사건 구조를 안정적으로 유지하지는 못한 것으로 해석하였다. 다음 문자 예측의 정확도가 높아지고 Training Loss가 감소하더라도, 생성된 문장이 반드시 문법적·의미적으로 자연스러워지는 것은 아님을 보여준다.

3.2 다음 문자 Top-5 예측 결과

학습 데이터에서 가져온 문맥인

오늘도 또 우리 수탉이 막 쫓 에 대한 결과는 다음과 같다.

*전체 1271개 문자 확률의 합: 1.0000

쫓기, **쫓는**, **쫓겨** 는 모두 가능한 연결이다. 모델이 무작위 문자가 아니라 현재 문맥에 자연스럽게 이어질 가능성이 있는 문자를 높은 순위로 배치하였음을 확인하였다.

순위	다음 문자	확률
1	기	0.640
2	는	0.284
3	겨	0.053
4	를	0.005
5	아	0.004

학습 corpus에 그대로 존재하지 않는 문맥인 **장인님은**

화가 나서 나를 예서는 공백의 확률이 0.998로 나타났다. 목적격 조사 **를** 다음에 띄어쓰기가 올 가능성을 높게 예측한 것으로, 모델이 문자 수준에서 한국어의 띄어쓰기 패턴을 학습했음을 보여준다.

순위	다음 문자	확률
1		0.998
2	된	0.001
3	\n	0.000
4	하	0.000
5	에	0.000

그러나 하나의 새로운 문맥에서 높은 확률이 나왔다는 사실만으로 일반화 성능이 충분하다고 판단할 수는 없다. 특히 공백 하나에 99.8%의 확률이 집중된 결과는 모델의 예측이 지나치게 확신에 차 있을 가능성도 보여준다. 다양한 검증 문장에 대한 추가 평가가 필요하다.

3.3 확인한 한계

1. 학습 데이터가 김유정의 단편 8편으로 제한되어 있다. 데이터의 양과 문체가 **한정**되어 있어 모델이 다양한 한국어 표현을 배우기 어렵고, 특정 작품의 표현을 반복하거나 암기할 가능성이 있다.
2. 최대 문맥 길이가 128문자이다. 생성문이 길어지면 초기 내용이 입력 범위에서 제외되므로 인물, 사건, 주제를 **장기간 일관되게 유지하기 어렵다**.
3. **문자 단위 토큰화**를 사용한다. Vocabulary가 단순하고 미등록 단어 문제가 적다는 장점이 있지만, 모델이 단어와 형태소의 의미를 직접적인 단위로 처리하지 못한다. 이로 인해 **답답답하게**, **아침마침** 과 같은 부자연스러운 문자 조합이 생성될 수 있다.
4. Training Loss만 측정하였으며 **Validation Loss는 측정하지 않았다**. 최종 Loss가 낮아졌다는 사실은 학습 데이터의 다음 문자를 잘 예측하게 되었다는 의미일 뿐, 새로운 문장을 잘 생성한다는 의미는 아니다.
5. 확률적 샘플링을 사용하였기 때문에 하나의 생성문만으로 **성능을 판단하기 어렵다**. 학습이 더 진행된 모델도 샘플링 결과에 따라 일시적으로 부자연스러운 문장을 만들 수 있다.

4. 성찰 및 개선 방향

4.1 실제 LLM과 미니 언어모델의 차이

구분	본 실습의 미니 언어모델	실제 LLM
토큰 단위	문자 단위	주로 서브워드 단위
파라미터 수	약 294만 개	수십억 개 이상 가능
학습 데이터	김유정 단편 8편	대규모-다분야 데이터
문맥 길이	최대 128문자	수천~수십만 토큰 가능
학습 과정	다음 문자 예측	사전학습, 지시학습, 정렬 등
주요 능력	짧은 문학 문제 생성	질의응답, 요약, 추론, 도구 사용 등

실제 LLM도 기본적으로 앞 문맥을 바탕으로 다음 토큰을 예측한다는 점에서는 본 모델과 원리가 같다. 그러나 실제 LLM은 훨씬 많은 파라미터와 데이터, 긴 문맥을 사용하며 사전학습 이후 지시학습과 정렬 과정을 추가로 수행한다. 따라서 미니 언어모델은 Transformer의 기본 원리를 관찰하기에는 적합하지만, 실제 LLM의 지식과 추론 능력을 그대로 재현하는 모델은 아니다.

4.2 실습을 통해 이해한 점

이번 실습을 통해 언어모델의 텍스트 생성이 미리 저장된 완성 문장을 꺼내는 과정이 아니라, 현재까지의 문맥을 바탕으로 다음 문자 확률을 반복적으로 계산하는 과정임을 이해하였다.

또한 Attention은 문장을 그대로 이해하는 하나의 기능이라기보다, 현재 위치에 필요한 앞 문맥의 정보를 선택하고 결합하는 계산 구조임을 확인하였다. Transformer Block을 여러 번 통과하면서 각 문자의 벡터에 문맥 정보가 누적되고, 최종적으로 Vocabulary 전체에 대한 다음 문자 점수가 계산된다.

가장 중요하게 이해한 점은 Loss 감소와 문장 이해를 동일하게 볼 수 없다는 것이다. Loss의 감소는 학습 데이터에서 다음 문자 패턴을 더 정확하게 예측하게 되었다는 뜻이다. 생성문의 의미적 자연스러움과 새로운 문장에 대한 일반화 능력은 별도의 검증이 필요하다.

4.3 향후 개선 방향

본 실습에서는 학습이 진행되면서 무작위 문자 배열이 실제 한국어 문장과 비슷한 형태로 발전하였다. 그러나 최종 생성문에서는 단어와 조사의 결합이 부자연스럽고, 앞뒤 사건의 의미가 충분히 연결되지 않는 한계가 나타났다. 이를 보완하기 위해 다음과 같은 추가 실험을 고려할 수 있다.

첫째, **학습 횟수에 따른 성능을 비교할 필요가 있다.** 현재 모델은 2,000회까지 학습하였지만, Training Loss만으로 학습이 충분했는지 판단하기는 어렵다. 논문을 찾아보니, 검증 데이터의 오차를 관찰하면서 과적합이 발생하기 전에 학습을 종료하는 Early Stopping 방법이 관련 연구에서 제안되었다 (Prechelt, 1998). 따라서 데이터를 Training Set과 Validation Set으로 나누고, 2,000회와 4,000회 학습 시점의 Training Loss와 Validation Loss를 비교할 수 있다. 두 값이 함께 감소한다면 추가 학습이 일반화 성능도 개선한 것으로 볼 수 있지만, Training Loss만 감소한다면 학습 문장을 암기하는 과적합이 발생한 것으로 해석할 수 있다.

둘째, **생성 설정에 따른 문장 품질을 비교할 필요가 있다.** 동일한 언어모델이라도 다음 토큰을 선택하는 방법에 따라 생성문의 자연스러움과 반복 정도가 달라질 수 있다. Holtzman et al.(2020)은 확률이 낮은 후보를 제한하고 누적 확률을 기준으로 후보를 선택하는 Nucleus Sampling을 제안하였다. 이를 참고하여 현재 설정인 `temperature=1.0`, `top_k=10` 과 두 설정을 낮춘 버전인 `temperature=0.7`, `top_p=0.9` 를 비교하고, 문법적 오류와 반복 표현이 감소하는지 확인할 수 있다.

추가 실험	비교 조건	확인할 내용
학습 횟수 비교	2,000회와 4,000회	Training Loss와 Validation Loss를 비교하여 추가 학습의 효과와 과적합 여부 확인
생성 설정 비교	현재 설정과 <code>temperature=0.7</code> , <code>top_p=0.9</code>	문법적 오류와 반복 표현이 감소하는지 생성문 비교

두 실험에서는 동일한 시작 문장을 사용하고, 각 조건에서 생성문을 3개씩 만들어 비교한다. 이를 통해 현재 생성문의 한계가 학습량 부족 때문인지, 생성 설정의 영향 때문인지 간단히 확인할 수 있을 것이다.

4.4 결론

결론적으로 본 실습에서는 약 294만 개의 파라미터를 가진 문자 단위 Transformer 모델을 김유정 단편 8편에 학습시켰으며, Training Loss가 7.1354에서 0.82754까지 감소하는 과정을 확인하였다. 학습 전에는 무작위 문자 배열이 생성되었으나, 학습 후에는 실제 한국어 단어, 띄어쓰기, 문장부호, 대화문 형식과 김유정 작품에서 자주 나타나는 어휘가 생성되었다. 따라서 모델이 학습 데이터의 문자 배열과 일부 문체적 특징을 학습했다는 점에서는 의미 있는 발전이 있었다.

그러나 최종 생성문은 한국어 문장의 외형에는 가까워졌지만, 단어와 조사의 결합이 부자연스럽고 인물, 행동, 사건 사이의 관계가 일관되게 유지되지 않았다. 이는 낮은 Training Loss가 곧바로 문장의 의미 이해나 새로운 데이터에 대한 일반화 능력을 보장하지는 않음을 보여준다.

현재 실험만으로는 이러한 한계가 2,000회의 학습 횟수 부족 때문인지, 작은 데이터와 문자 단위 토큰화, 128자의 문맥 길이, 모델 용량 등의 구조적 조건 때문인지 확인할 수 없다. 따라서 향후에는 작품 단위의 검증 데이터 구성, 학습 횟수 비교, 문맥 길이와 토큰화 방식 비교, 생성 설정별 반복 평가 등과 같은 추가 실험을 수행하면 더욱 정확한 분석이 가능 할 것으로 보인다.

본 실습은 완성도 높은 문학 작품을 생성하는 데에는 도달하지 못했지만, Transformer가 무작위 상태에서 문자 패턴과 문장 형식을 점진적으로 학습하는 과정과 그 한계를 실제 수치와 생성문을 통해 확인했다는 점에서 의미가 있다. 또한 언어모델은 Training Loss 하나만으로 평가할 수 없으며, 일반화 성능, 의미 연결성, 다양성, 원문 암기 여부와 계산 비용을 함께 고려해야 한다는 점을 이해하였다.

참고문헌

- Prechelt, L. (1998). *Early Stopping—But When?* In *Neural Networks: Tricks of the Trade*.
- Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). *The Curious Case of Neural Text Degeneration*. ICLR 2020.